SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

Authentication System 03

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

회원정보 수정

비밀번호 변경

비밀번호 암호화

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

참고

- 비밀번호 초기화
- PasswordChangeForm 인자 순서
- Auth built-in form 코드

학습 시작

내 비밀번호는 안전할까?

- 우리의 비밀번호는 어떻게 저장되고 있을까요?
- 그리고 만약 우리가 만든 서비스가 해킹당해 DB가 통째로 유출된다면 어떻게 될까요?
- 개발자로서, 이 최악의 상황을 어떻게 막아야 할까요?

1. 비밀번호는 우리가 입력한 값으로 저장되고 있을까요?
2. 해커가 무수히 많은 시도를 통해 비밀번호를 해킹하려고 하면 어떻게 할까요?

➢ 비밀번호 암호화 과정을 살펴보고, 암호화에 대한 기초 지식을 알 수 있어요.

➢ 다양한 공격 시도에 대한 방어 기술을 살펴볼 수 있어요.

학습 목표

- ✓ UserChangeForm을 상속받아 회원정보 수정 폼을 만든다.
- ✓ PasswordChangeForm을 사용하여 비밀번호 변경 기능을 구현한다.
- ✓ update_session_auth_hash 함수로 비밀번호 변경 후 세션을 유지한다.
- ✓ 단방향 해시를 통한 비밀번호 암호화의 필요성을 안다.
- ✓ 솔트와 키 스트레칭을 통한 암호화 강화 원리를 안다.
- ✓ Django의 비밀번호 저장 방식(알고리즘, 솔트 등)을 안다.
- ✓ Django 내장 auth URL을 활용해 비밀번호 초기화를 구현한다.

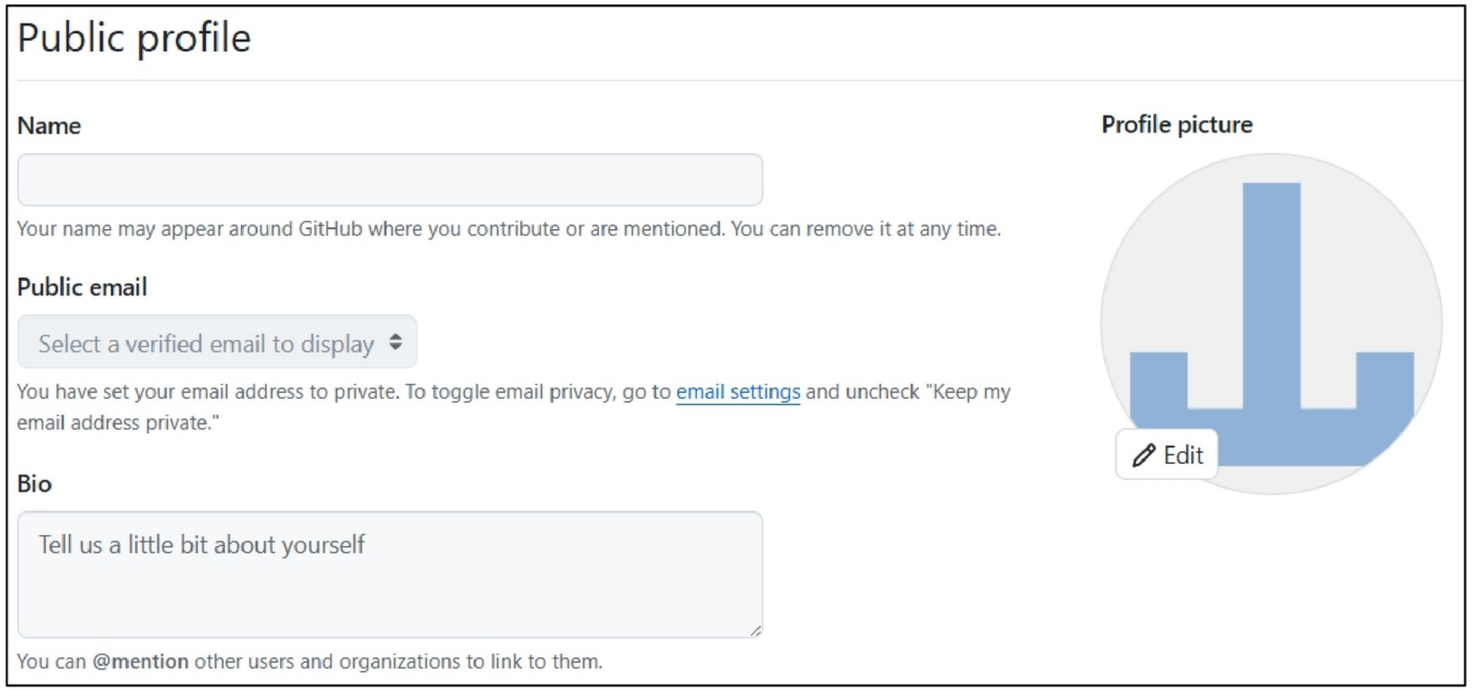
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

회원정보 수정

회원정보 수정

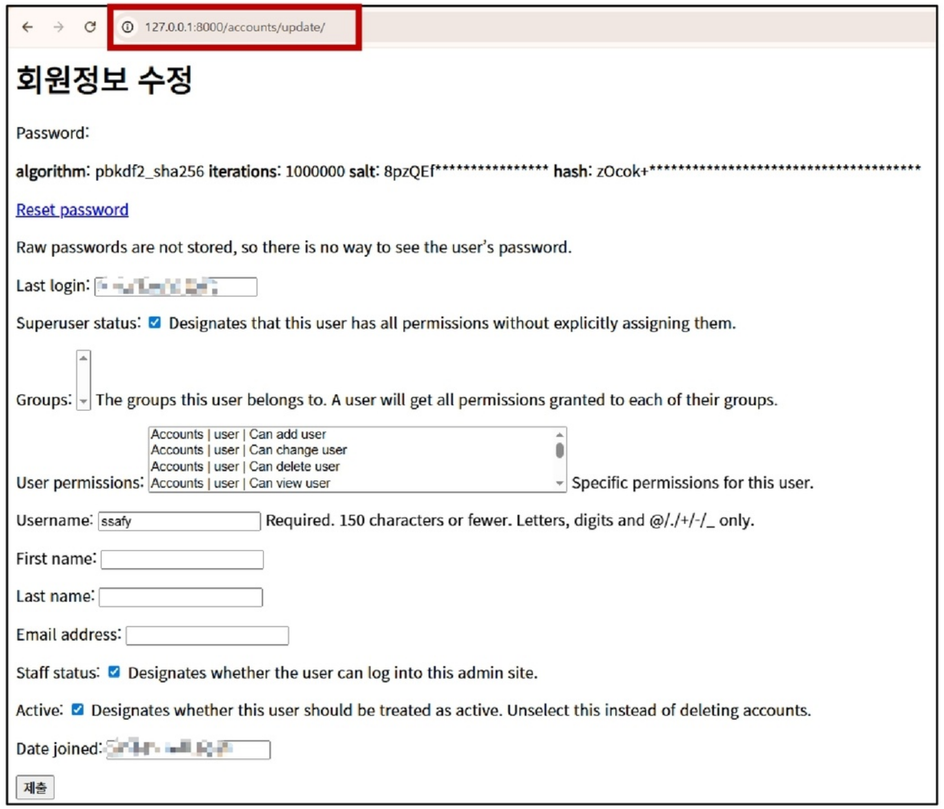
- User 객체를 Update 하는 과정
- 수정할 대상 User 객체를 가져오고, 입력받은 새로운 정보로 기존 내용을 갱신



<그림1_Django_Authentication 03_Github 회원정보 수정 예시>

회원정보 수정 기능 구현 (1/5)

- 회원정보 수정 경로 url 생성



<그림2_Django_Authentication 03_회원정보 수정 페이지>

```python
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    ...,
    path('update/', views.update, name='update'),
]
```

UserChangeForm()

- 회원정보 수정 시 사용자 입력 데이터를 받는 built-in ModelForm
- ModelForm이기 때문에, 유효성 검사를 통과한 데이터로 기존 User 객체의 내용을 갱신하고 저장 (주로 관리자 페이지에서 사용)

```
class UserChangeForm(forms.ModelForm):
    password = ReadOnlyPasswordHashField(
        label=_("Password"),
        help_text=_(
            "Raw passwords are not stored, so there is no way to see "
            "the user’s password."
        ),
    )

    class Meta:
        model = User
        fields = "__all__"
        field_classes = {"username": UsernameField}
```

<그림3_Django_Authentication 03_UserChangeForm>

회원정보 수정 기능 구현 (2/5)

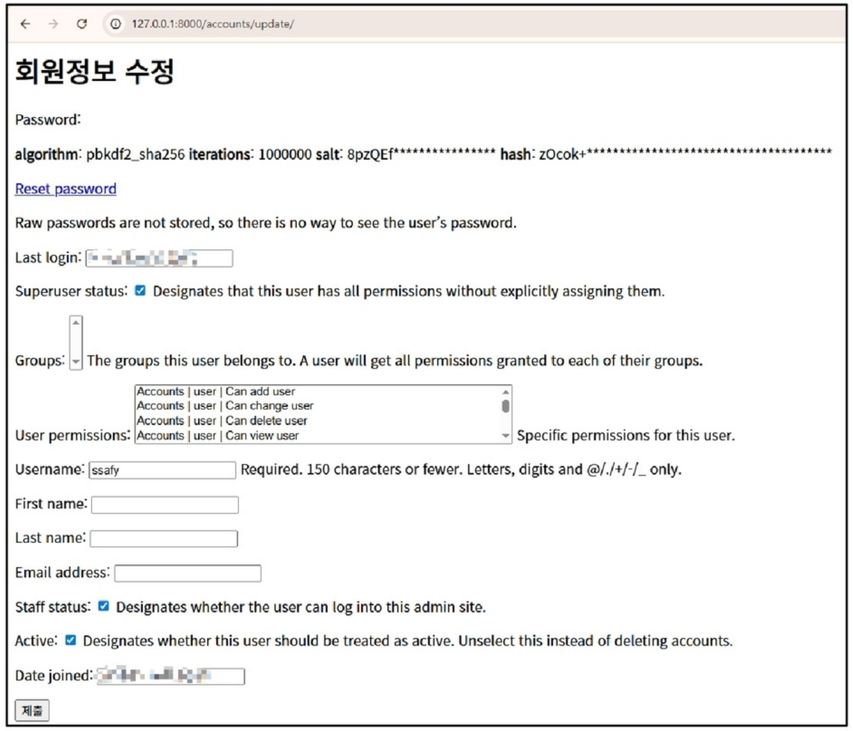
- [복습] 커스텀 유저 모델을 사용 하기 위해서 Form을 다시 작성
- Custom User model을 사용할 수 있도록 상속 후 일부분만 재작성

```
# accounts/forms.py
from django.contrib.auth import get_user_model
from django.contrib.auth.forms import UserChangeForm


class CustomUserChangeForm(UserChangeForm):
    class Meta(UserChangeForm.Meta):
        model = get_user_model()
```

https://docs.djangoproject.com/en/5.2/topics/auth/customizing/#custom-users-and-the-built-in-auth-forms

회원정보 수정 기능 구현 (3/5)

- .../accounts/update/ url로 요청이 들어왔을 때 실행할 update 함수 작성
- 회원정보 수정에 사용할 데이터를 입력 받는 CustomUserChangeForm built-in form 사용

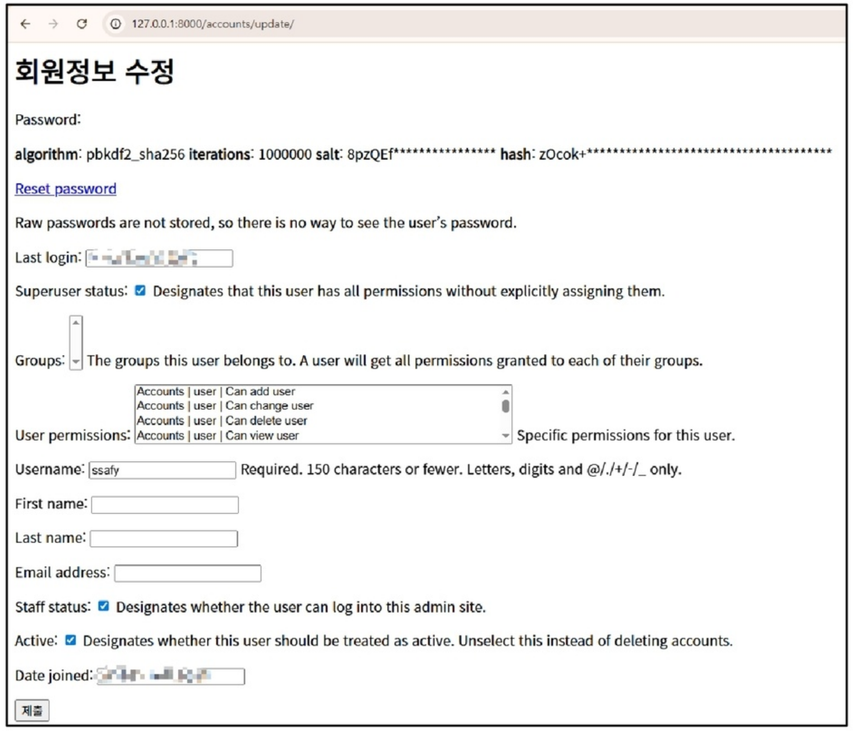


<그림2_Django_Authentication 03_회원정보 수정 페이지>

```python
# accounts/views.py

from .forms import CustomUserChangeForm


def update(request):
    if request.method == 'POST':
        pass
    else:
        form = CustomUserChangeForm(instance=request.user)
    context = {
        'form': form,
    }
    return render(request, 'accounts/update.html', context)
```

회원정보 수정 기능 구현 (4/5)

- 회원정보 수정은 User 객체를 갱신하는(서버 상태를 바꾸는) 요청이므로 POST 방식을 사용
- 서버로부터 전달받은 `CustomUserChangeForm`을 화면에 출력

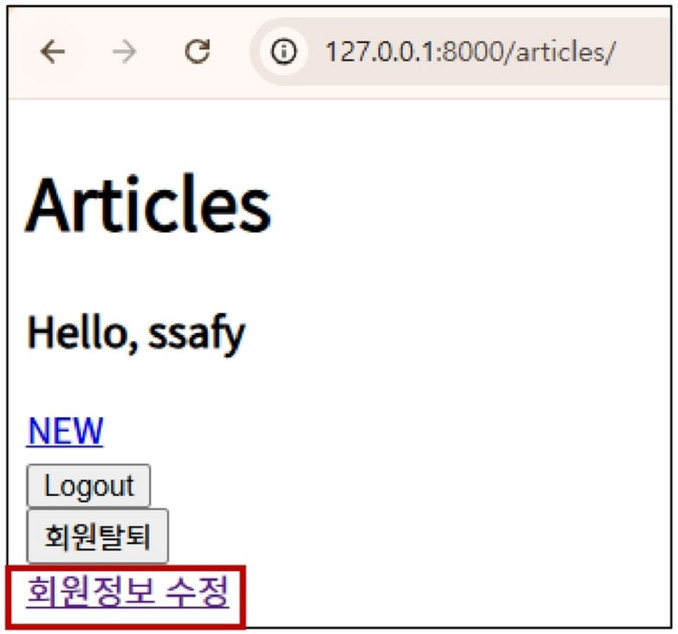


<그림2_Django_Authentication 03_회원정보 수정 페이지>

```html
<!-- accounts/update.html -->

<h1>회원정보 수정</h1>
<form action="{% url 'accounts:update' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

회원정보 수정 기능 구현 (5/5)

- 메인 페이지에서 회원정보 수정 페이지에 접근할 수 있는 태그 생성



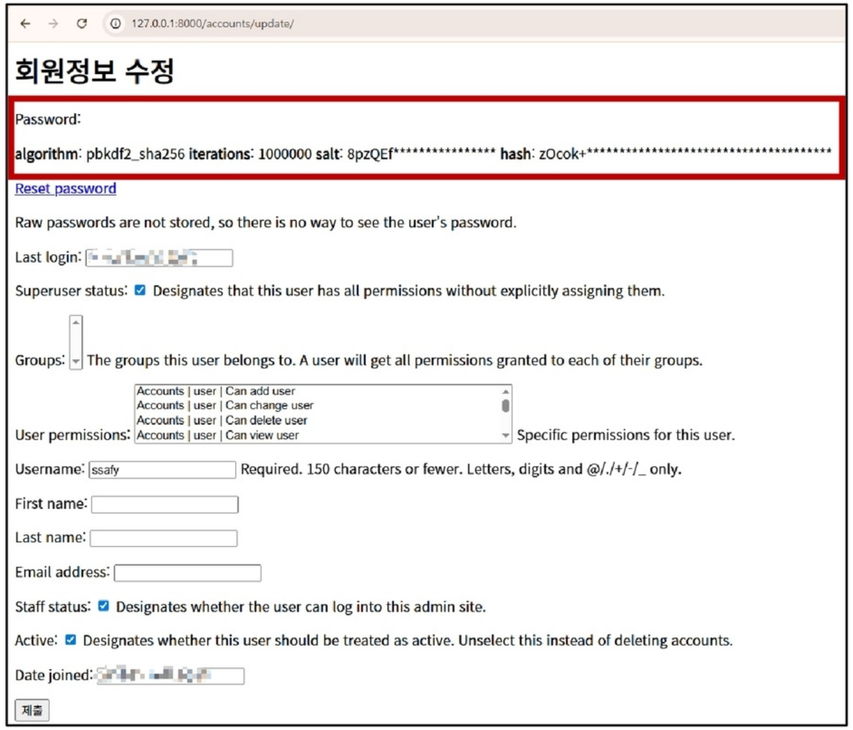
<그림4_Django_Authentication 03_메인 페이지 회원정보 수정 링크>

```
<!-- articles/index.html -->

<a href="{% url 'accounts:update' %}">회원정보 수정</a>
```

UserChangeForm 사용 시 문제점

- User 모델의 모든 정보들(fields)까지 모두 출력됨
- 일반 사용자들이 접근하면 안되는 정보는 출력하지 않아야 함

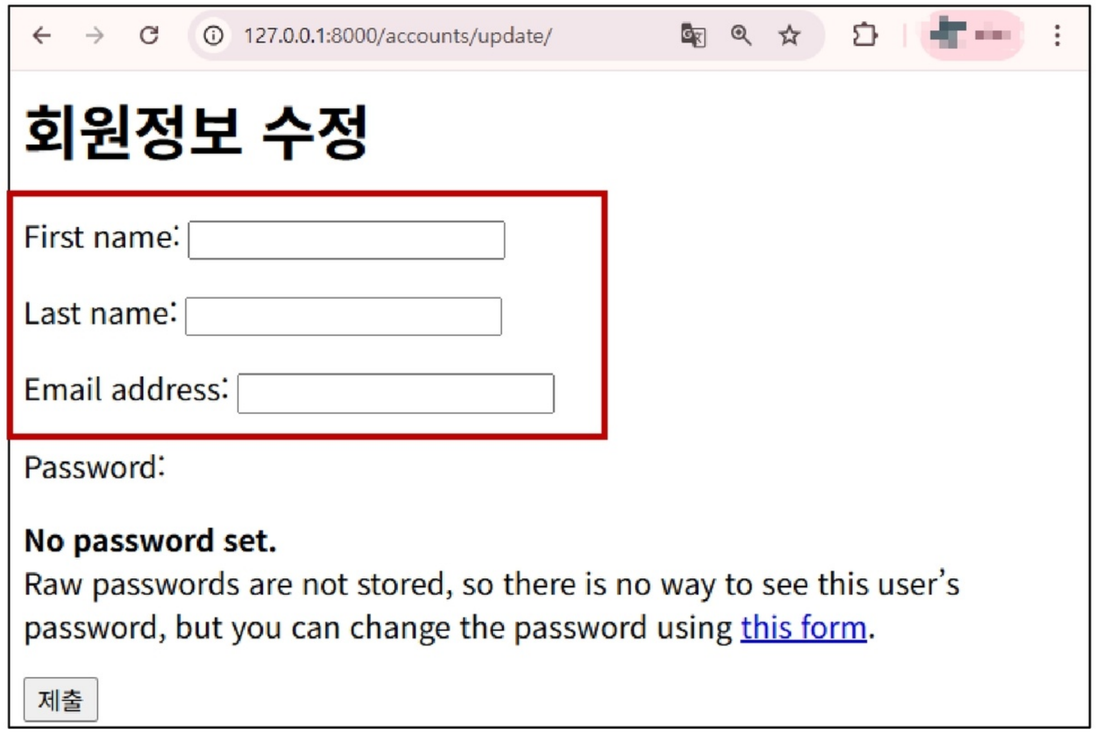
➢ CustomUserChangeForm에서 출력 필드를 다시 조정하기



<그림2_Django_Authentication 03_회원정보 수정 페이지>

CustomUserChangeForm 출력 필드 재정의

- User Model의 필드 목록 확인



<그림5_Django_Authentication 03_회원정보 수정 필수 필드 페이지>

```python
# accounts/forms.py

class CustomUserChangeForm(UserChangeForm):

    class Meta(UserChangeForm.Meta):
        model = get_user_model()
        fields = ('first_name', 'last_name', 'email',)
```

https://docs.djangoproject.com/en/5.2/ref/contrib/auth/

회원정보 수정 로직 완성

```python
# accounts/views.py

@login_required
def update(request):
    if request.method == 'POST':
        form = CustomUserChangeForm(request.POST, instance=request.user)
        # form = CustomUserChangeForm(data=request.POST, instance=request.user)
        if form.is_valid():
            form.save()
            return redirect('articles:index')
    else:
        form = CustomUserChangeForm(instance=request.user)
    context = {
        'form': form,
    }
    return render(request, 'accounts/update.html', context)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

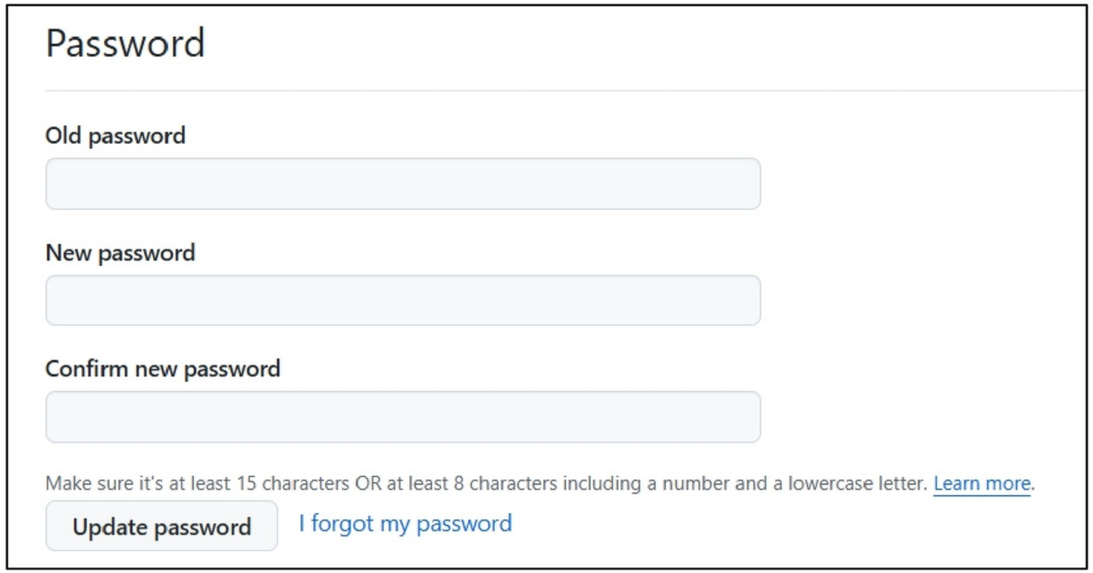
비밀번호 변경

비밀번호 변경

- 비밀번호 변경은 User 객체의 password 필드를 Update 하는 과정

  (기존 비밀번호로 본인을 확인한 뒤 새 비밀번호를 해시해서 저장)

- 기존 비밀번호를 통해 사용자를 인증하고, 새로운 비밀번호를 암호화하여 갱신



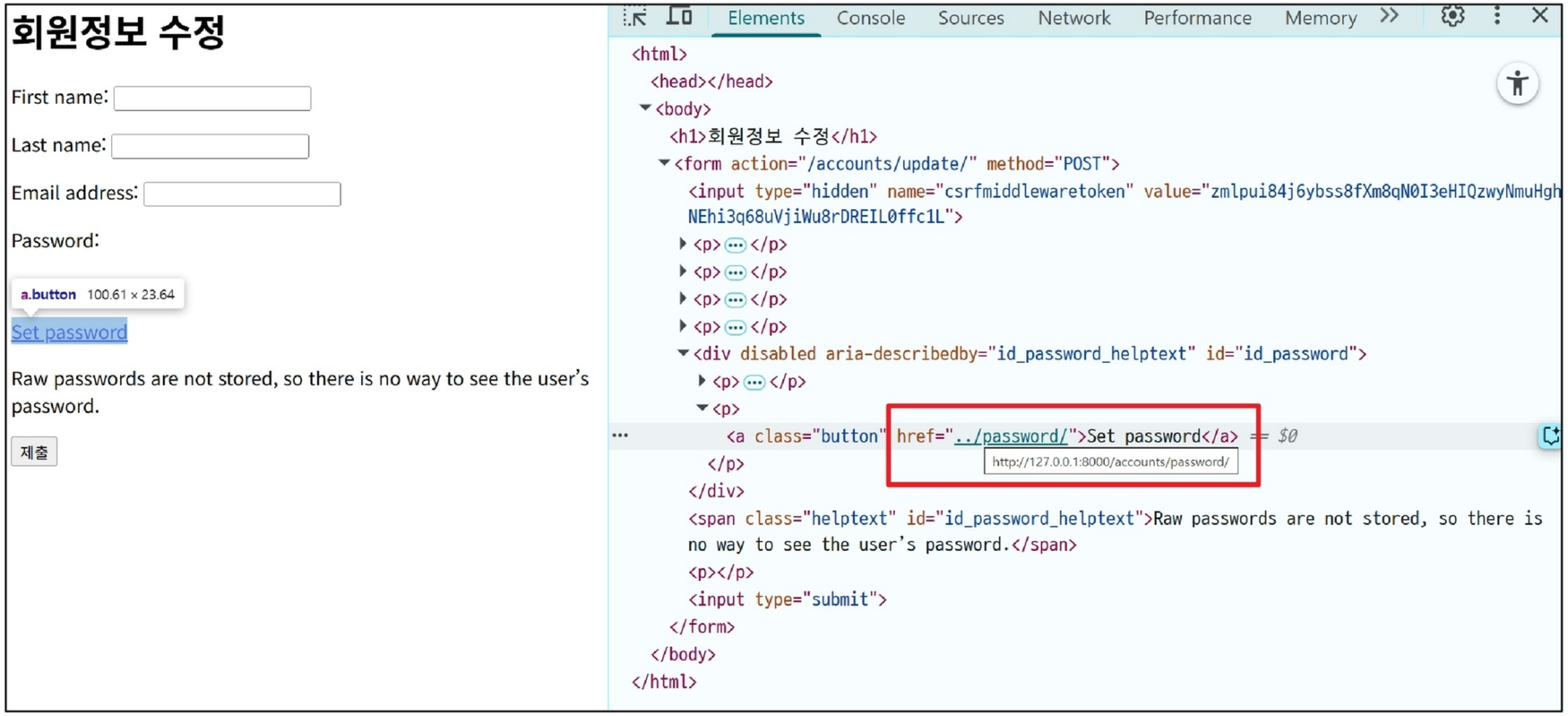
<그림6_Django_Authentication 03_Github 비밀번호 변경 예시>

session
서버 측에서 생성되어
클라이언트와 서버 간의 상태를
유지,
상태 정보를 저장하는 데이터
저장 방식

<개념1_Django_Authentication 03_세션>

비밀번호 변경 기능 구현 작성 (1/5)

- django는 비밀번호 변경 페이지를 회원정보 수정 form 하단에서 별도 주소로 안내



<그림7_Django_Authentication 03_비밀번호 변경 링크>

※ django에서 기본 경로로 /accounts/password/를 안내함

비밀번호 변경 기능 구현 작성 (2/5)

- Django에서 안내하는 비밀번호 변경 URL에 맞춰서 작성

```
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    ...,
    path('password/', views.password, name='password'),
]
```

비밀번호 변경 기능 구현 작성 (3/5)

- .../accounts/password/ url로 요청이 들어올 때 실행할 password 함수 작성
- 비밀번호 변경에 사용할 데이터를 입력 받는 PasswordChangeForm built-in form 사용

```
# accounts/views.py

from django.contrib.auth.forms import PasswordChangeForm

@login_required
def password(request):
    if request.method == 'POST':
        pass
    else:
        form = PasswordChangeForm(request.user)
    context = {
        'form': form,
    }
    return render(request, 'accounts/password.html', context)
```

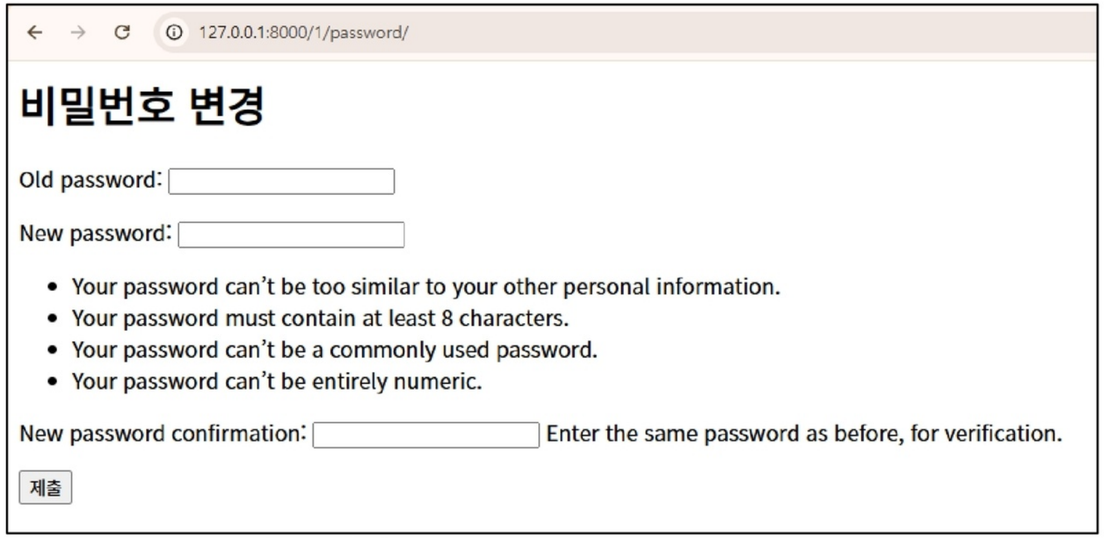
PasswordChangeForm()

- 비밀번호 변경 시 사용자 입력 데이터를 받는 built-in Form
- 일반 'Form'이며, 유효성 검사(기존 비밀번호 확인, 새 비밀번호 일치 여부)를 통과한 데이터로 사용자의 비밀번호를 안전하게 암호화하여 갱신하는 역할을 수행

```python
class PasswordChangeForm(SetPasswordForm):
    """
    A form that lets a user change their password by entering their old
    password.
    """

    error_messages = {
        **SetPasswordForm.error_messages,
        "password_incorrect": _(
            "Your old password was entered incorrectly. Please enter it again."
        ),
    }
    old_password = forms.CharField(
        label=_("Old password"),
        strip=False,
        widget=forms.PasswordInput(
            attrs={"autocomplete": "current-password", "autofocus": True}
        ),
    )
```

<그림9_Django_Authentication 03_PasswordChangeForm>

비밀번호 변경 기능 구현 작성 (4/5)



<그림8_Django_Authentication 03_비밀번호 변경 페이지>

```
<!-- accounts/password.html -->

<h1>비밀번호 변경</h1>
<form action="{% url 'accounts:password' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

비밀번호 변경 기능 구현 작성 (5/5)

```python
# accounts/views.py

@login_required
def password(request):
    if request.method == 'POST':
        form = PasswordChangeForm(request.user, request.POST)
        # form = PasswordChangeForm(user=request.user, data=request.POST)
        if form.is_valid():
            user = form.save()
            return redirect('articles:index')
    else:
        form = PasswordChangeForm(request.user)
    context = {
        'form': form,
    }
    return render(request, 'accounts/password.html', context)
```

세션 무효화 방지

암호 변경 시 세션 무효화

- 비밀번호가 변경되면 기존 세션과의 회원 인증 정보가 일치하지 않게 되어 버려 로그인 상태가 유지되지 못하고 로그아웃 처리됨
- Django는 세션에 비밀번호에서 만든 값(해시)을 함께 기록해 둠. 비밀번호가 바뀌면 이 값이 달라져서, 기존 세션을 "더 이상 유효하지 않음"으로 판단 (다른 기기에서 로그인된 세션도 함께 로그아웃 되는 보안 효과)

➢ 암호 변경 시 세션 무효화를 막는 방법은?

암호 변경 시 세션 무효화를 막아주는 함수 - "update_session_auth_hash"

update_session_auth_hash(request, user)

- 암호가 변경되면 새로운 password의 Session Data로 기존 session을 자동으로 갱신
- update_session_auth_hash를 password 함수에 적용

```
# accounts/views.py

from django.contrib.auth import update_session_auth_hash

@login_required
def password(request):
    if request.method == 'POST':
        form = PasswordChangeForm(request.user, request.POST)
        if form.is_valid():
            user = form.save()
            update_session_auth_hash(request, user)
            return redirect('articles:index')
    …
```

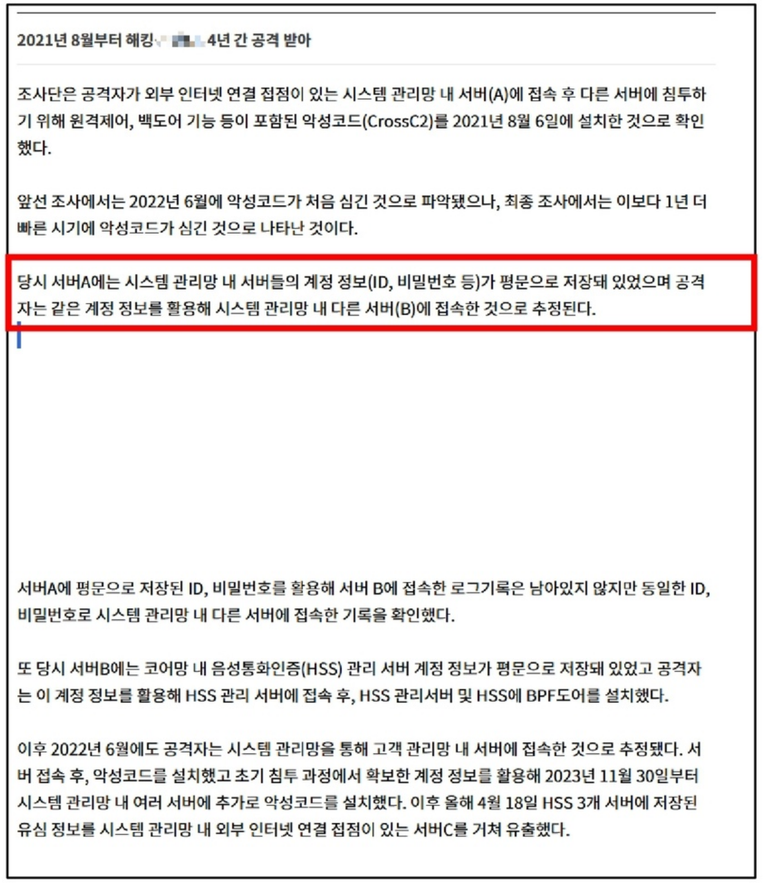
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

비밀번호 암호화

암호화의 중요성

- 많은 해킹사태가 발생하고 있고, 데이터가 유출되더라도 그 내용을 알 수 없도록 하는 암호화는 특히 중요



2021년 8월부터 해킹 [redacted] 4년 간 공격 받아

조사단은 공격자가 외부 인터넷 연결 접점이 있는 시스템 관리망 내 서버(A)에 접속 후 다른 서버에 침투하기 위해 원격제어, 백도어 기능 등이 포함된 악성코드(CrossC2)를 2021년 8월 6일에 설치한 것으로 확인했다.

앞선 조사에서는 2022년 6월에 악성코드가 처음 심긴 것으로 파악됐으나, 최종 조사에서는 이보다 1년 더 빠른 시기에 악성코드가 심긴 것으로 나타난 것이다.

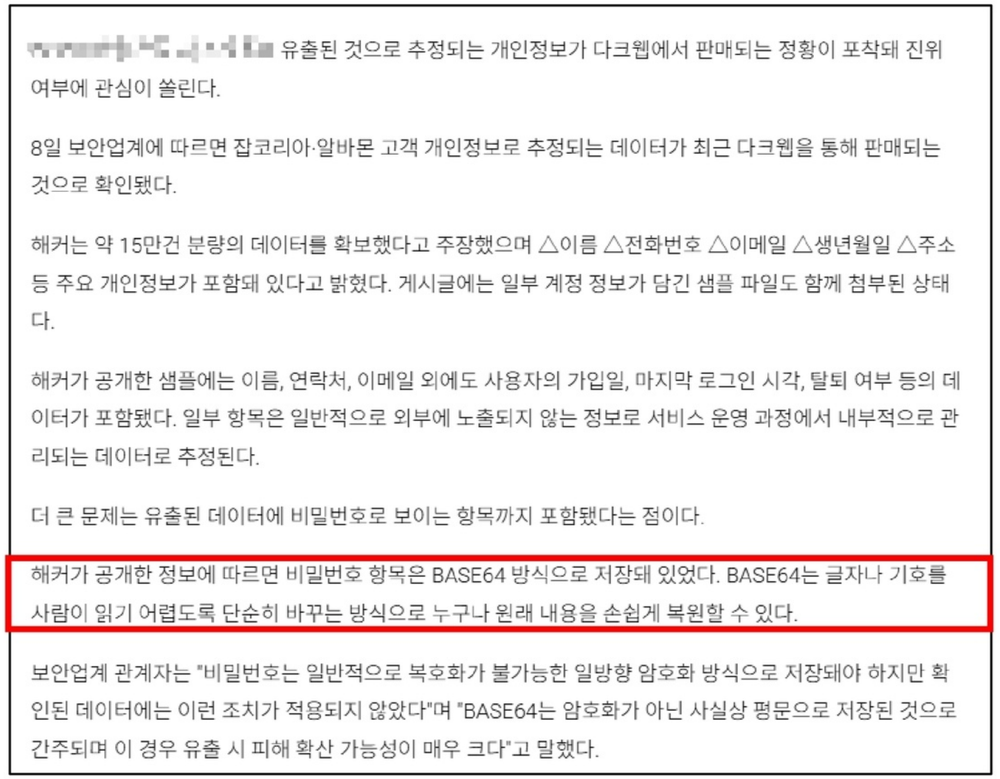
당시 서버A에는 시스템 관리망 내 서버들의 계정 정보(ID, 비밀번호 등)가 평문으로 저장돼 있었으며 공격자는 같은 계정 정보를 활용해 시스템 관리망 내 다른 서버(B)에 접속한 것으로 추정된다.

서버A에 평문으로 저장된 ID, 비밀번호를 활용해 서버 B에 접속한 로그기록은 남아있지 않지만 동일한 ID, 비밀번호로 시스템 관리망 내 다른 서버에 접속한 기록을 확인했다.

또 당시 서버B에는 코어망 내 음성통화인증(HSS) 관리 서버 계정 정보가 평문으로 저장돼 있었고 공격자는 이 계정 정보를 활용해 HSS 관리 서버에 접속 후, HSS 관리서버 및 HSS에 BPF도어를 설치했다.

이후 2022년 6월에도 공격자는 시스템 관리망을 통해 고객 관리망 내 서버에 접속한 것으로 추정됐다. 서버 접속 후, 악성코드를 설치했고 초기 침투 과정에서 확보한 계정 정보를 활용해 2023년 11월 30일부터 시스템 관리망 내 여러 서버에 추가로 악성코드를 설치했다. 이후 올해 4월 18일 HSS 3개 서버에 저장된 유심 정보를 시스템 관리망 내 외부 인터넷 연결 접점이 있는 서버C를 거쳐 유출했다.

<그림10_Django_Authentication 03_해킹 피해 사례1>



[redacted] 유출된 것으로 추정되는 개인정보가 다크웹에서 판매되는 정황이 포착돼 진위 여부에 관심이 쏠린다.

8일 보안업계에 따르면 잡코리아·알바몬 고객 개인정보로 추정되는 데이터가 최근 다크웹을 통해 판매되는 것으로 확인됐다.

해커는 약 15만건 분량의 데이터를 확보했다고 주장했으며 △이름 △전화번호 △이메일 △생년월일 △주소 등 주요 개인정보가 포함돼 있다고 밝혔다. 게시글에는 일부 계정 정보가 담긴 샘플 파일도 함께 첨부된 상태다.

해커가 공개한 샘플에는 이름, 연락처, 이메일 외에도 사용자의 가입일, 마지막 로그인 시각, 탈퇴 여부 등의 데이터가 포함됐다. 일부 항목은 일반적으로 외부에 노출되지 않는 정보로 서비스 운영 과정에서 내부적으로 관리되는 데이터로 추정된다.

더 큰 문제는 유출된 데이터에 비밀번호로 보이는 항목까지 포함됐다는 점이다.

해커가 공개한 정보에 따르면 비밀번호 항목은 BASE64 방식으로 저장돼 있었다. BASE64는 글자나 기호를 사람이 읽기 어렵도록 단순히 바꾸는 방식으로 누구나 원래 내용을 손쉽게 복원할 수 있다.

보안업계 관계자는 "비밀번호는 일반적으로 복호화가 불가능한 일방향 암호화 방식으로 저장돼야 하지만 확인된 데이터에는 이런 조치가 적용되지 않았다"며 "BASE64는 암호화가 아닌 사실상 평문으로 저장된 것으로 간주되며 이 경우 유출 시 피해 확산 가능성이 매우 크다"고 말했다.

<그림11_Django_Authentication 03_해킹 피해 사례2>

우리가 사용하는 비밀번호는 어떻게 저장되고 있을까? (1/3)

1. 사용자가 입력한 비밀번호 **그대로** 저장하는 방식 => **보안에 매우 취약하므로 X**

- 데이터베이스가 해킹 당하면, 공격자는 아이디와 비밀번호 목록을 그대로 손에 넣게 되고,

  이를 통해 직접 로그인하여 개인정보, 금융 정보, 주소록 등 모든 데이터를 유출하거나 서비스를 악용할 수 있음

- 악의적인 내부 직원이 데이터베이스에 접근하여 모든 사용자의 비밀번호를 볼 수 있음

- 대부분의 사람들은 여러 서비스에서 동일한 아이디와 비밀번호를 사용하기에 탈취한 정보를 이용해 다른 사이트

  에 그대로 대입하여 2차 피해를 발생시킴(Credential Stuffing 공격)

우리가 사용하는 비밀번호는 어떻게 저장되고 있을까? (2/3)

2. 일정한 규칙에 따라 비밀번호를 알아볼 수 없는 문자로 '인코딩' 한 후 저장 => **보안에 매우 취약하므로 X**

- 데이터베이스에 알아볼 수 없는 문자로 저장되어 있다고 하더라도,

  인코딩은 비밀키 없이도 정해진 규칙에 따라 누구나 원래의 값으로 되돌릴 수 있음

- 공격자는 아주 간단한 디코딩 작업만으로 모든 사용자의 실제 비밀번호를

  즉시 알아낼 수 있음

- 사실상 비밀번호를 평문으로 저장하는 것과 동일한 수준의 위험을 초래

인코딩
정보를 표현하는 형식을 다른 형식으로 변환하는 과정

<개념2_Django_Authentication 03_인코딩>

비밀키
암호화된 데이터를 풀거나, 디지털 서명을 만들 때 쓰는 열쇠

< 개념3_Django_Authentication 03_비밀키>

우리가 사용하는 비밀번호는 어떻게 저장되고 있을까? (3/3)

3. 비밀번호를 복원이 불가능한 고정된 길이의 문자열로 변환 후 저장 => 보안에 필수

- 데이터베이스가 유출되어도 공격자는 복잡하게 얽힌 문자열을 보게 되고, 복원이 불가능하기 때문에 실제 비밀번호를 알 수 없음
- 악의적인 내부 직원이 비밀번호를 보더라도 암호화된 비밀번호를 보게 되므로, 실제 비밀번호를 유추할 수 없음

➢ 비밀번호를 복원이 불가능한 고정된 길이로 바꾸는 과정을 "해시(hash)"라고 함

TIP

왜 고정된 길이의 문자열로 변환하나요?

1. **보안성:** 변환된 문자열의 길이가 다르다면, 길이만 보고도 원래 비밀번호의 길이를 유추할 수 있습니다.
2. **일관성:** 길이가 동일하기 때문에 저장 공간을 예측하고 설계하기 쉬우며, 검색/비교하는 처리 속도도 일정하게 유지할 수 있습니다.

해시

해시 (Hash)

임의의 크기를 가진 데이터를

고정된 크기의 **고유한 값**으로 변환하는 것

"해시는 데이터를 믹서기에 넣어 만든 주스와 같습니다.“

- 아무리 복잡한 과일(데이터)이라도 한번 갈아버리면, 절대 다시 원래의 과일로 되돌릴 수 없습니다. (단방향성)
- 하지만 같은 재료를 넣으면 언제나 똑같은 맛의 주스가 나옵니다.
- 이처럼 원본을 알 수 없게 변환하여 보안을 지키거나, 데이터가 변경되지 않았는지 확인하는 기술입니다.

TIP

엄밀히 말하면 암호화(Encryption)는 열쇠(키)가 있으면 원래대로 되돌릴 수 있는 양방향 변환이고, 해시(Hash)는 되돌릴 수 없는 단방향 변환입니다. 비밀번호는 되돌릴 필요가 없기 때문에 해시로 저장합니다. 일상적으로는 둘 다 "암호화"라고 부르기도 합니다.

해시 함수

해시함수 (Hash function)

임의 길이 데이터를 입력 받아

고정 길이의 값(보통 16진수 문자열로 표시)으로 변환해 주는 함수

데이터를 넣으면 암호화된 문자열(해시값)을 뱉어내는 '마법의 상자'

1. **고정된 길이:** 아무리 긴 소설책을 넣어도, 짧은 단어를 넣어도 결과물의 길이는 똑같습니다.
2. **단방향성 (One-way):** 결과물만 보고 원본을 추측하는 것은 불가능합니다.
3. **눈사태 효과 (Avalanche Effect):** 입력 값이 점 하나만 달라져도, 결과물은 완전히 다른 값이 됩니다.

해시(hash)

- 데이터를 고정된 크기의 값으로 변환하는 과정
- 작은 변화에도 해시값이 크게 달라지는 특성으로 인해 변조 여부를 쉽게 확인할 수 있음
- 입력값이 들어오더라도 해시 함수에 의해서 다른 값으로 바뀌며, 동일한 값은 항상 동일한 해시값을 생성

해시

<그림12_Django_Authentication 03_해시 과정>

Django에서의 암호화

- Django는 기본적으로 SHA-256을 바탕으로 한 PBKDF2 방식(pbkdf2_sha256)으로 비밀번호를 저장 (PBKDF2는 뒤에서 자세히)
- 입력한 비밀번호의 길이와는 상관없이 동일한 길이의 해시 값을 생성
- 1글자만 다르더라도 전혀 다른 해시 값을 생성
- 작성한 비밀번호
  - 1: 1234
  - 2: 1235
  - 3: 12341234

id INTEGER	* password varchar(128)
1	pbkdf2_sha256$600000$U7e5KT0ENtZKIA0xoWCWxs$cBWsUin9jHRdQppftp5pH06ltcWLZWeQajSZ54JjDIM=
2	pbkdf2_sha256$600000$hLTECde6TpeEkv6qepn6d8$JyTHluTIOuWh8S+KjRFwjM0l8s3qLoQjA6UpuunGH44=
3	pbkdf2_sha256$600000$32L7Li84kXRB6qtssWFV7e$TLni0ZiQAEigdxDJb/813mOWJGb3LGPg4L0yzO90ZgU=

<그림13_Django_Authentication 03_암호화 저장 예시>

SHA-256

SHA-256

Secure Hash Algorithm - 256

현재 가장 널리 사용되는
표준 암호화 해시 알고리즘

어떤 데이터를 넣더라도 항상 256비트(64글자)의 결과물을 만들어냅니다.

현재 가장 신뢰받는 알고리즘으로, 비트코인의 핵심 기술이자 전 세계 보안 표준으로 사용됩니다.

(해시값에서 원본을 거꾸로 계산해 내는 것은 사실상 불가능합니다. 하지만 계산이 매우 빠르다는 점이 비밀번호 저장에서는 약점이 됩니다.)

해시 함수를 활용해 단방향 암호화를 하면 더 이상 문제가 없을까?

- 비밀번호를 해시 값으로 저장하면 공격자가 유출된 데이터베이스를 봐도 원래 비밀번호를 알 수 없으니 안전해 보일 수 있음
- 하지만 공격자들은 해시 값을 미리 계산해두는 방식으로 공격을 시도

➢ 이 방식이 바로 ‘레인보우 테이블’ 공격

레인보우 테이블(Rainbow Table)

- 공격자가 자주 사용되는 비밀번호들(예: 123456, qwer1234, ...)을 미리 수백만, 수십억 개를 해시로 변환해 저장해 둔 거대한 정답지
- 공격 방식
  1. 공격자가 DB를 탈취해 사용자의 비밀번호 해시 값을 얻음
  2. 해시 값을 공격자 자신의 레인보우 테이블에서 검색
  3. 테이블에서 일치되는 값을 찾아내면서, 역방향으로 되돌리지 않고 비밀번호를 알아내는 데 성공

미리 계산된 비밀번호	미리 계산된해시값 (SHA256)
123456	8d969eef...
qwer1234	4d4f2636...
iloveyou	e4ad93ca...
...수억 개...	...
password1234	b9c95064...

DB에서 훔친 해시 값:
1c8eda66...

비밀번호:
iloveyou

<그림14_Django_Authentication 03_레인보우 테이블 공격 예시>

그러면 레인보우 테이블 공격은 어떻게 방어할까?

- 공격자가 아무리 거대한 레인보우 테이블을 가지고 있더라도, 사용자의 해시 값이 레인보우 테이블에 없도록 해시 값을 만들면 됨
- 해시 값은 입력값이 단 한 글자만 달라져도 해시값은 완전히 달라지는 눈사태 효과가 있음
- 결국 같은 비밀번호라도 사용자마다 "임의의 문자열"을 비밀번호에 붙여서 해시 암호화를 진행
  - 임의의 문자열이 추가된 상태로 해시 값을 만들기 때문에 눈사태 효과에 의해 같은 비밀번호도 다른 해시 값이 나옴

➢ 여기서 "임의의 문자열" 역할을 하는 것이 바로 "솔트(Salt)"

솔트(Salt)

- 각 사용자마다 고유하게 생성된 임의의 문자열(솔트)을 비밀번호에 덧붙여서 해시 값을 생성
- 이 솔트(Salt)는 해시 값과 함께 데이터베이스에 저장

사용자	비밀번호	해시 값
User A	qwer1234	5e884898...
User B	qwer1234	5e884898...

솔트 X

동일

<그림15_Django_Authentication 03_솔트 미사용 예시>

사용자	비밀번호	솔트	해시 값
User A	qwer1234	!@#$	a1b2c3d4...
User B	qwer1234	%^&*	x7y8z9w0...

솔트 O

동일하지 않음

<그림16_Django_Authentication 03_솔트 사용 예시>

TIP

공격자가 솔트(Salt)값을 알아도 되나요?

- 공격자가 솔트(Salt)값을 알아도 상관없습니다.
- userA의 솔트와 해시값을 훔쳤다면, 공격자는 userA만을 위한 레인보우 테이블을 만들어야 합니다.
- 이는 '하나의 답안지로 수백만 명을 공격하는 방식'에서 '한 명을 공격하기 위해 매번 새로운 답안지를 만드는 방식'으로 바꾸면서, 공격의 효율성을 극도로 떨어뜨립니다.

솔트(Salt)로 레인보우 테이블 공격을 막았으니, 안전할까요?

- 공격자는 이제 미리 만들어 둔 답안지를 쓸 수 없어짐
- 그래서 단순하지만 확실한 방법으로써 가능한 모든 비밀번호를 하나씩 직접 대입 해보는 방식으로 공격
- 이 방식은 현대 컴퓨터의 엄청난 속도 때문에 생각보다 훨씬 위협적
  - 최신 GPU는 초당 수백억 번 이상 추측할 수 있음

➢ 이 방식이 바로 “무차별 대입 공격(Brute-force Attack)”

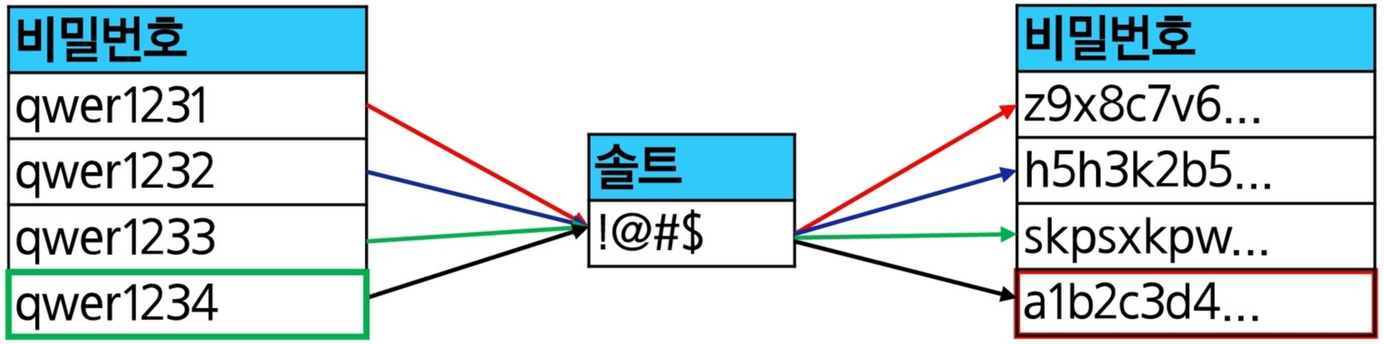
무차별 대입 공격(Brute-force Attack)

- 가장 원시적이지만 강력한 방법으로써 가능한 모든 비밀번호를 하나씩 대입하는 방식
- 이 공격은 시간과의 싸움이며, 현대 컴퓨터의 빠른 연산 속도가 공격자의 무기가 됨
- 탈취한 사용자 계정 정보

사용자	비밀번호	솔트	해시 값
User A	qwer1234	!@#$	a1b2c3d4...

<그림17_Django_Authentication 03_탈취 사용자 예시>

- 탈취 과정



<그림18_Django_Authentication 03_무차별 대입 공격 예시>

그러면 무차별 대입 공격(Brute-force Attack)은 어떻게 막을까요?

- 공격자는 현대 컴퓨터의 빠른 연산 속도를 무기로 공격을 시도하고 있음
- 결국 해결책은 "연산 속도를 늦추는 것"이 핵심
- 연산 속도를 늦추기 위해서 의도적으로 비밀번호 검증 과정을 느리게 만듦
  - 느리게 만들기 위해서 의도적으로 해시 연산을 수십만 번 반복시켜, 공격 속도를 늦춤

➢ 이 방식이 바로 "키 스트레칭(Key Stretching)"

TIP

연산 속도를 늦추면 사용자가 로그인할 때 느리다는 불편을 겪을 수도 있지 않을까요?
- **사용자는 실제로 체감하지 못합니다.**
- **해시 연산 속도를 1개당 0.2초로 바꾼다고 하면, 실제로 공격자는 1초에 5번의 공격만 가능합니다.**
- **하지만 사용자에게 0.2초는 느리다고 하기엔 충분히 빠른 시간입니다.**

키 스트레칭(Key Stretching)

- 솔트(Salt)를 적용한 해시 함수를 수만 ~ 수십만 번 반복하여 연산 시간을 의도적으로 늘리는 기법
- '속도'가 무기인 무차별 대입 공격(Brute-force Attack)을 방어할 수 있음
- Django는 이 키 스트레칭을 구현하기 위해서 PBKDF2 라는 검증된 알고리즘을 기본으로 사용
  - 최근에는 더 강력한 보안을 제공하는 `Argon2, bcrypt` 같은 알고리즘도 지원

※ **PBKDF2, Argon2, bcrypt 알고리즘은 키 스트레칭을 구현하는 다양한 방법 정도로만 이해하기**

구분	키 스트레칭 없음 (1회)	키 스트레칭 적용 (100만 회)
1개 검증 시간	0.000001초	약 0.1~1초
10억 개 검증	약 17분	약 3.2년 이상

<표1_Django_Authentication 03_키 스트레칭 비교>

Django에서의 비밀번호 암호화

How Django stores passwords ¶

Django provides a flexible password storage system and uses PBKDF2 by default.

The **password** attribute of a **User** object is a string in this format:

```
<algorithm>$<iterations>$<salt>$<hash>
```

<그림19_Django_Authentication 03_Django 비밀번호 암호화 예시>

1. <algorithm> : 어떤 알고리즘을 쓰는 지
2. <iterations>: 키 스트레칭 횟수
3. <salt>: 생성된 솔트
4. <hash>: 생성된 최종 해시

https://docs.djangoproject.com/en/5.2/topics/auth/passwords/

비밀번호 암호화 정리

- 암호화 과정을 이해한 후, 검증된 프레임워크의 보안 기능을 신뢰하고 사용하기
  - 보안은 매우 어렵고 복잡한 분야이며, Django 공식에도 가급적 재발명하지 않을 것을 권장하고 있음
- 단순히 코드를 복사해서 붙여넣는 것을 넘어, 이 기능이 '왜' 이렇게 만들어졌는지 이해하면, 더 견고하고 안전한 애플리케이션을 만들 수 있음
- 사용자는 우리의 서비스를 믿고 소중한 개인정보를 맡기고, 그들의 데이터를 안전하게 지키는 것은 개발자의 '가장 기본적인 책임이자 직업윤리'

https://docs.djangoproject.com/en/5.2/topics/auth/passwords/

참고

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

비밀번호 초기화

비밀번호 초기화

- 비밀번호를 잊어버린 사용자가 이메일을 활용하여 비밀번호를 다시 설정하는 과정
- 비밀번호 초기화 과정
  1. 비밀번호를 찾으려고 하는 이메일 입력
  2. 이메일로 비밀번호 재설정 링크를 전송
  3. 비밀번호 재설정 페이지에서 새로운 비밀번호 설정
  4. 초기화 후 다시 로그인

1. 비밀번호를 찾으려고 하는 이메일 입력 (1/3)

- 이메일을 입력하는 페이지를 직접 만들어야 할까?
  - ➢ **No!**

- Django는 비밀번호 관련 기능을 내장 view와 URL (django.contrib.auth.urls)로 제공하고 있음 (화면은 admin 앱의 기본 템플릿을 빌려 씀)

Authentication Views

Django provides several views that you can use for handling login, logout, and password management. These make use of the stock auth forms but you can pass in your own forms as well.

Django provides no default template for the authentication views. You should create your own templates for the views you want to use. The template context is documented in each view, see All authentication views.

Using the views

There are different methods to implement these views in your project. The easiest way is to include the provided URLconf in **django.contrib.auth.urls** in your own URLconf, for example:

```
urlpatterns = [
    path("accounts/", include("django.contrib.auth.urls")),
]
```

This will include the following URL patterns:

```
accounts/login/ [name='login']
accounts/logout/ [name='logout']
accounts/password_change/ [name='password_change']
accounts/password_change/done/ [name='password_change_done']
accounts/password_reset/ [name='password_reset']
accounts/password_reset/done/ [name='password_reset_done']
accounts/reset/<uidb64>/<token>/ [name='password_reset_confirm']
accounts/reset/done/ [name='password_reset_complete']
```

<그림20_Django_Authentication 03_Django auth example>

https://docs.djangoproject.com/en/5.2/topics/auth/default/#module-django.contrib.auth.views

1. 비밀번호를 찾으려고 하는 이메일 입력 (2/3)

- Django에서 제공하는 비밀번호 관련 기능을 활용하기 위해 `django.contrib.auth.urls` 를 crud/urls.py에 포함

```python
# crud/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path('articles/', include('articles.urls')),
    path('accounts/', include('accounts.urls')),
    path('accounts/', include('django.contrib.auth.urls')),
]
```

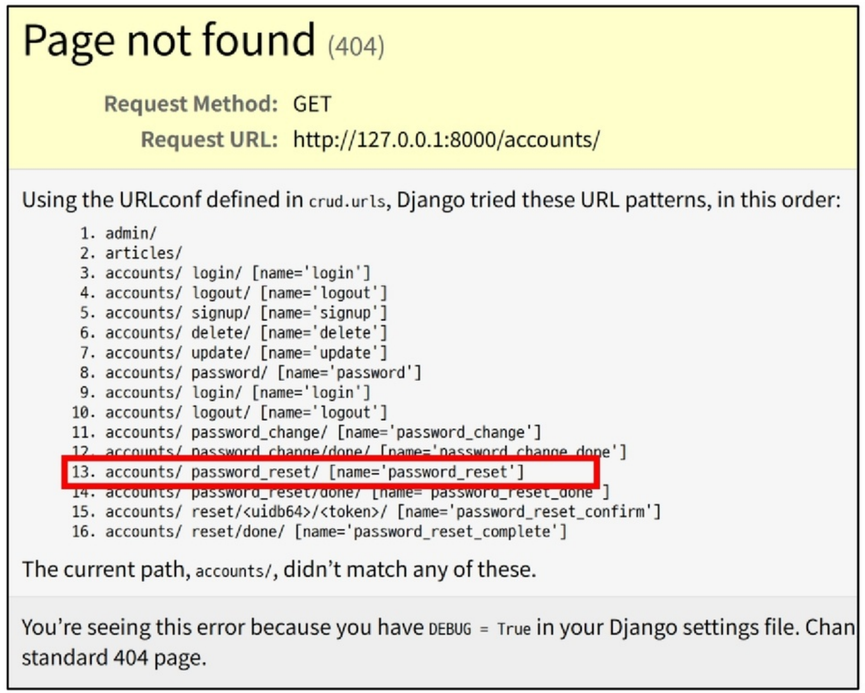
TIP

같은 accounts에 2개의 include가 사용되어도 괜찮나요?

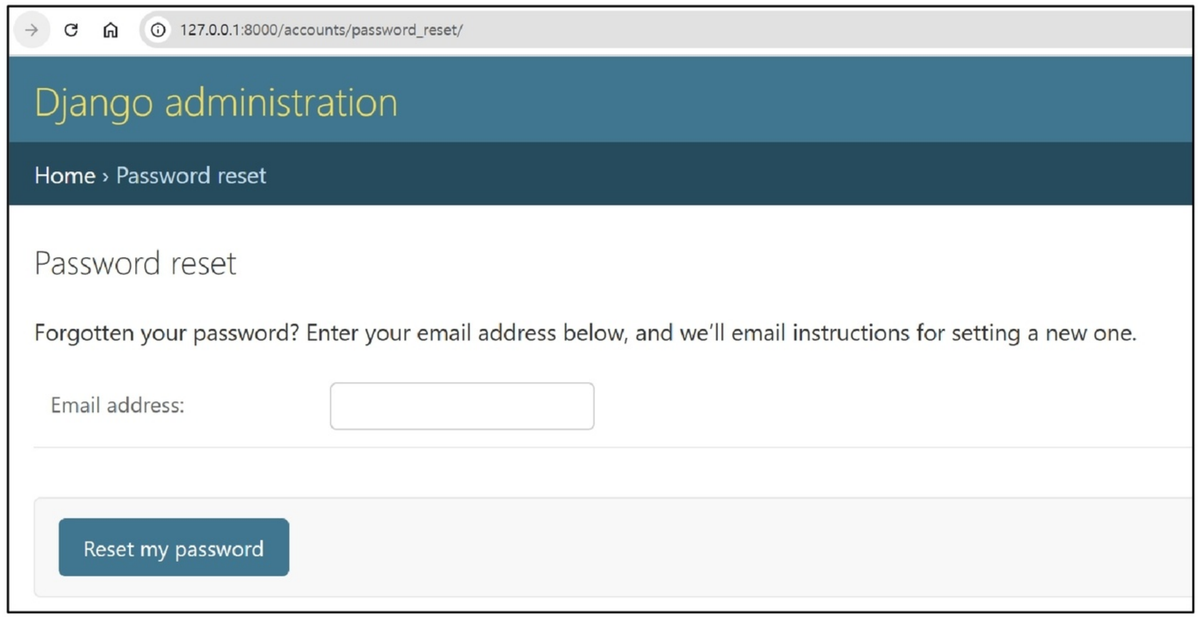
- 같은 prefix로 여러 번 include하면 위에서부터 순서대로 찾습니다. `login/`, `logout/`처럼 겹치는 경로는 먼저 적은 accounts.urls가 우선 처리됩니다. 그래서 우리 앱의 include를 먼저 적어야 합니다.

1. 비밀번호를 찾으려고 하는 이메일 입력 (3/3)

- http://127.0.0.1:8000/accounts/ 입력해보고, 제공하는 url 목록을 확인
- 제공되는 url 목록에서 'password_reset' 경로를 입력해보고, 비밀번호 초기화 페이지로 접근



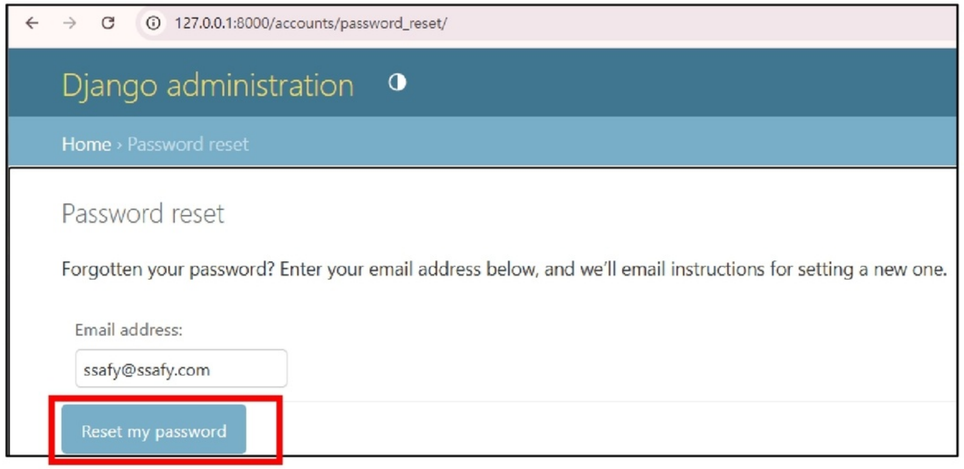
<그림21_Django_Authentication 03_비밀번호 찾기 페이지 목록>



<그림22_Django_Authentication 03_비밀번호 찾기 이메일 입력창>

2. 이메일로 비밀번호 재설정 링크를 전송 (1/4)

- 비밀번호를 찾으려고 하는 이메일을 작성하고 'Reset my password' 버튼을 눌러 이메일 전송



<그림23_Django_Authentication 03_비밀번호 찾기 이메일 입력>

※ 입력한 이메일에 매칭되는 사용자 이메일이 없다면 이메일 기능은 동작하지 않음

TIP

왜 입력한 이메일에 매칭되는 사용자 이메일이 없다면 이메일 기능이 동작하지 않나요?

- 에러나 주의 문구가 나타나면 사용자 입장에서는 편리하겠지만, 임의의 사용자가 이메일의 존재 여부를 확인하는 용도로 악용하는 사례를 막기 위해서 동작하지 않습니다.

2. 이메일로 비밀번호 재설정 링크를 전송 (2/4)

- 이메일 관련 설정을 따로 하지 않았기 때문에 이메일 전송 실패

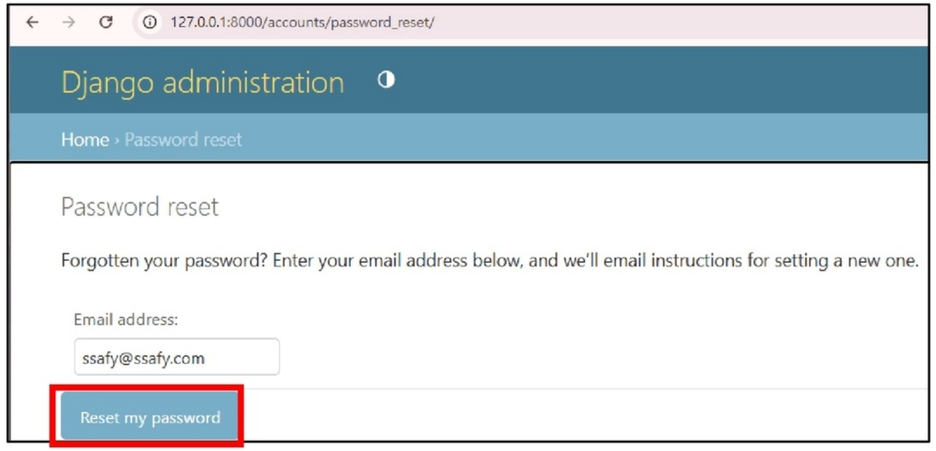
```
Failed to send password reset email to 1
Traceback (most recent call last):
  [redacted]
ConnectionRefusedError: [WinError 10061] 대상 컴퓨터에서 연결을 거부했으므로 연결하지 못했습니다
[08/Oct/2026 16:46:53] "POST /accounts/password_reset/ HTTP/1.1" 302 0
[08/Oct/2026 16:46:53] "GET /accounts/password_reset/done/ HTTP/1.1" 200 3638
```

<그림24_Django_Authentication 03_이메일 전송 실패 페이지>

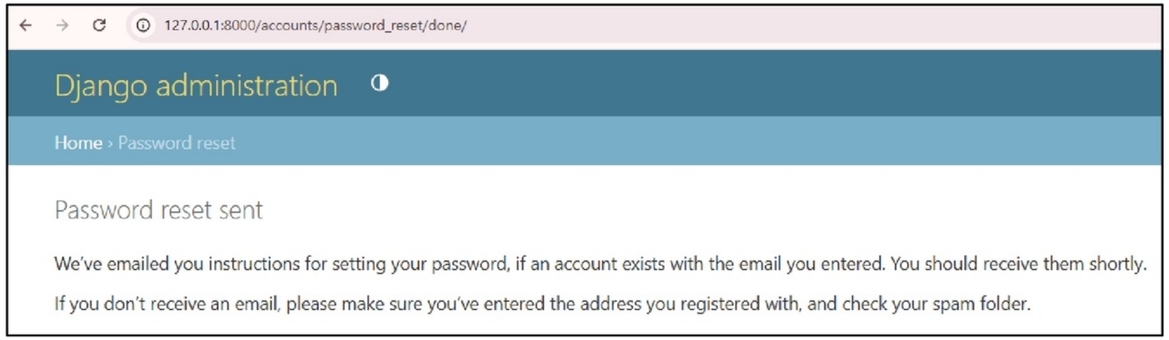
2. 이메일로 비밀번호 재설정 링크를 전송 (3/4)

- Django에서는 이메일을 보낸 내용을 콘솔에서 볼 수 있는 기능을 제공
- settings.py에서 **EMAIL_BACKEND** 세팅 후 다시 이메일 전송하기

```
# crud/settings.py

EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```



<그림25_Django_Authentication 03_에러 수정 후 이메일 입력 페이지>



<그림26_Django_Authentication 03_이메일 전송 완료 후>

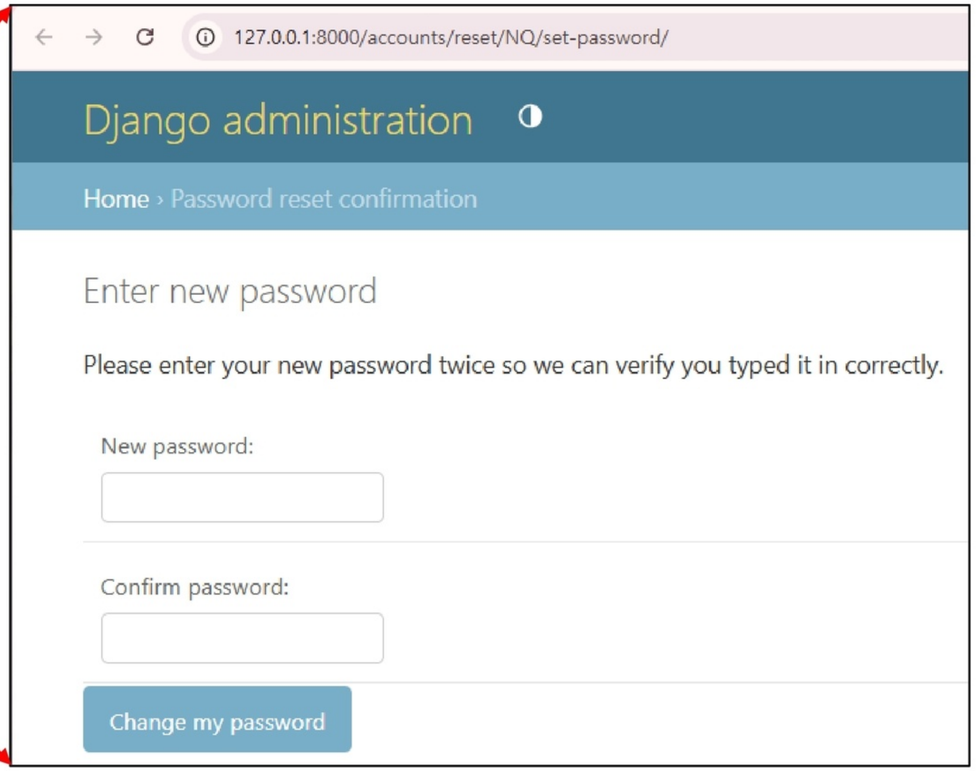
https://docs.djangoproject.com/en/5.2/topics/email/#console-backend

2. 이메일로 비밀번호 재설정 링크를 전송 (4/4)

- 콘솔창에 비밀번호 초기화 설정 페이지 안내 이메일을 확인하고, 해당 링크 들어가기

```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
Subject: Password reset on 127.0.0.1:8000
From: webmaster@localhost
To: ssafy@ssafy.com
Date: Thu, 08 Oct 2026 07:49:09 -0000
Message-ID: <179144574921.13472.11399961849935342189@JunhoKim>


You're receiving this email because you requested a password reset for your user account at 127.0.0.1:8000.

Please go to the following page and choose a new password:

http://127.0.0.1:8000/accounts/reset/MQ/dg52px-9de0bd3a87305eb827f258b873fcce58/

In case you've forgotten, you are: admin

Thanks for using our site!

The 127.0.0.1:8000 team
```

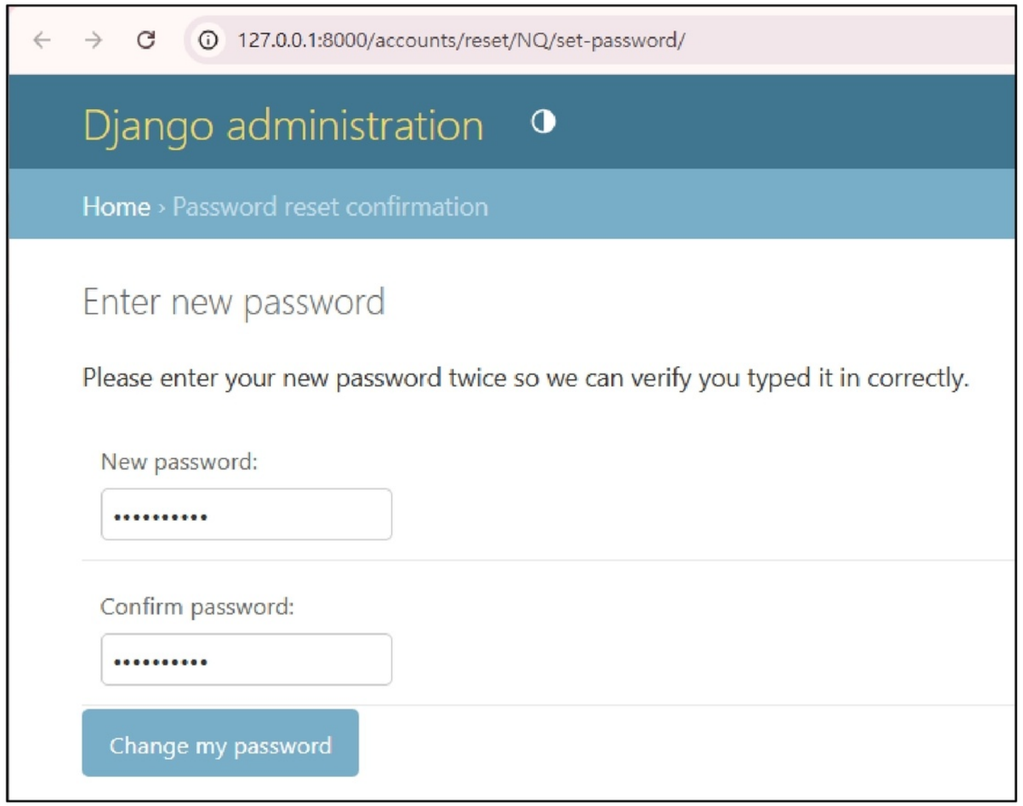
<그림27_Django_Authentication 03_콘솔 로그 이메일 내용>



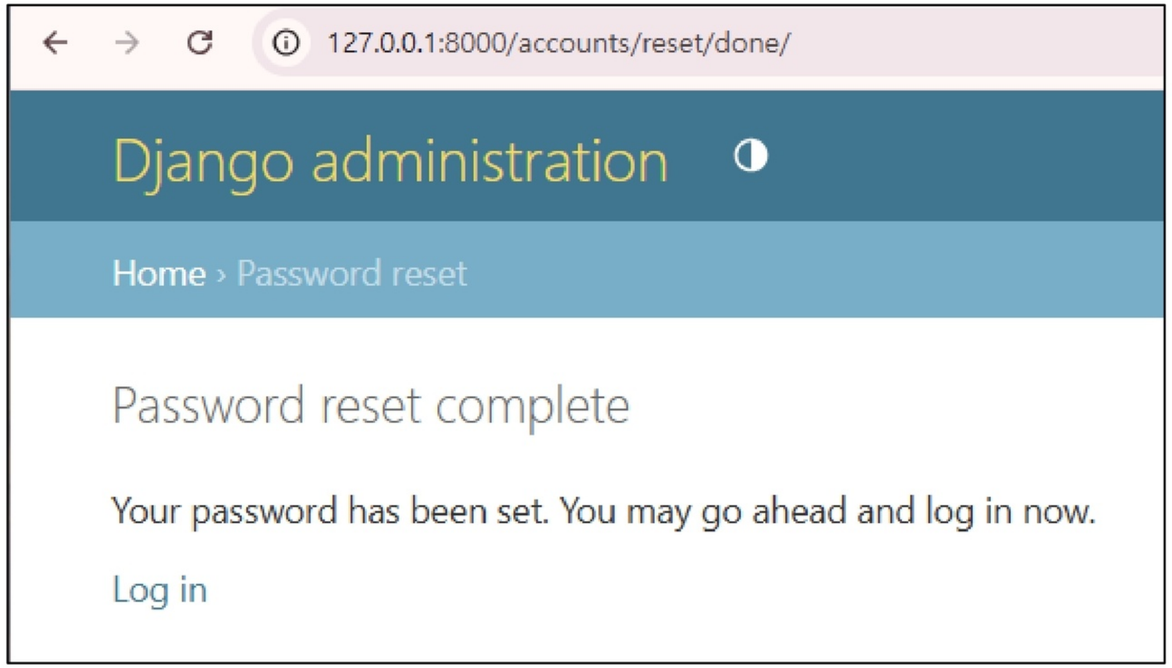
<그림28_Django_Authentication 03_새 비밀번호 입력 페이지>

3. 비밀번호 재설정 페이지에서 새로운 비밀번호 설정

- 새로운 비밀번호 설정 후 , 비밀번호 설정 완료 페이지 보여주기



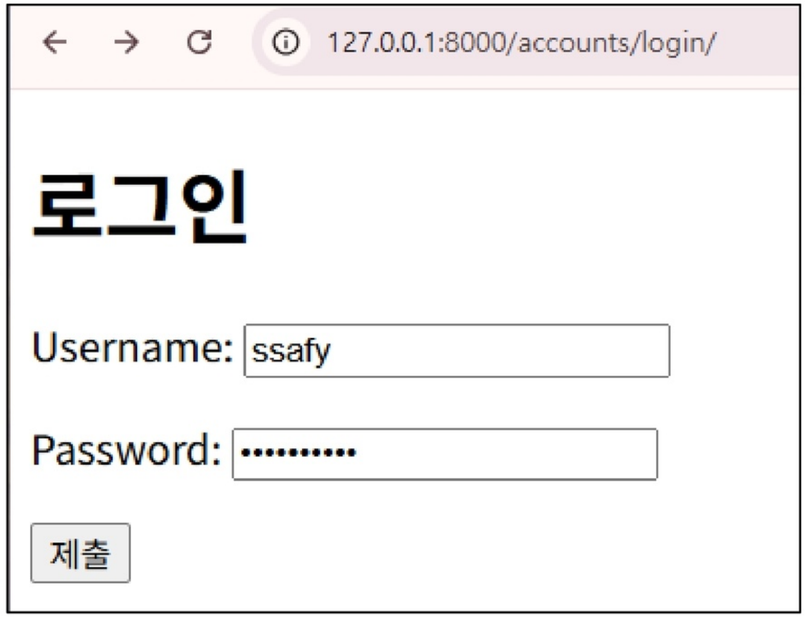
<그림29_Django_Authentication 03_새 비밀번호 입력 완료>



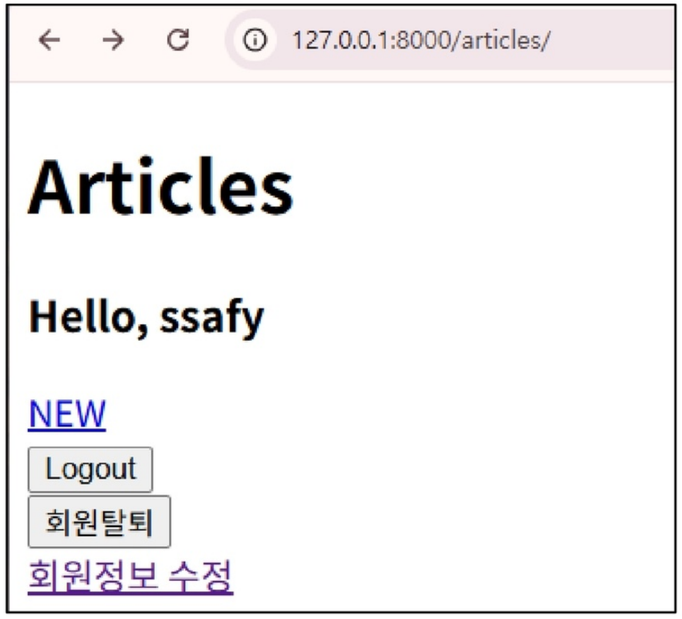
<그림30_Django_Authentication 03_비밀번호 변경 완료>

4. 초기화 후 다시 로그인

- 변경한 비밀번호로 로그인



<그림31_Django_Authentication 03_로그인 페이지>



<그림32_Django_Authentication 03_로그인 성공 후 페이지>

Django 에서는 수많은 '완성된 기능 모듈'을 제공

- 단순하게 '비밀번호 초기화' 기능을 구현한 것이 아니라, 잘 만들어진 모듈을 활용하는 법을 학습한 것
  - `django.contrib.auth.urls` 외에도 `admin`, `sessions`, `sitemaps` 등 다양한 기능을 제공
  - 이메일 서버가 없어도 테스트가 가능한 것처럼, 실제 서비스 환경을 흉내 내는 많은 모듈을 제공하고 있음
- 이런 다양한 모듈을 활용해서 개발 기술을 숙련하는 데 활용

PasswordChangeForm 인자 순서

PasswordChangeForm의 인자 순서

- PasswordChangeForm이 다른 Form과 달리 user 객체를 첫번째 인자로 받는 이유
  - 부모 클래스인 SetPasswordForm의 생성자 함수 구성을 따르기 때문

```python
class PasswordChangeForm(SetPasswordForm):
    """
    A form that lets a user change their password by entering their old
    password.
    """

    error_messages = {
        **SetPasswordForm.error_messages,
        "password_incorrect": _(
            "Your old password was entered incorrectly. Please enter it again."
        ),
    }
    old_password = forms.CharField(
        label=_("Old password"),
        strip=False,
        widget=forms.PasswordInput(
            attrs={"autocomplete": "current-password", "autofocus": True}
        ),
    )
```

<그림9_Django_Authentication 03_PasswordChangeForm>

https://github.com/django/django/blob/5.2/django/contrib/auth/forms.py#L531

부모 클래스에 따른 인자 구성

Form	부모 클래스	첫 번째 인자	사용 예
ArticleForm	ModelForm	data	ArticleForm(request.POST)
CustomUserChangeForm	ModelForm	data (+ instance)	CustomUserChangeForm(request.POST, instance=request.user)
AuthenticationForm	Form	request	AuthenticationForm(request, request.POST)
PasswordChangeForm	SetPasswordForm (Form)	user	PasswordChangeForm(request.user, request.POST)

TIP

부모 클래스가 다르면 생성자 함수의 인자 구성도 다르다.
헷갈리면 공식 문서나 소스 코드의 __init__을 확인하는 습관을 들이자.

Auth built-in form 코드

Auth built-in form github 코드

- 아래 링크에 들어가서 각 Form의 구조를 살펴보자
- `UserChangeForm()`
  - https://github.com/django/django/blob/5.2/django/contrib/auth/forms.py#L292
- `PasswordChangeForm()`
  - https://github.com/django/django/blob/5.2/django/contrib/auth/forms.py#L422

실습

회원정보 수정

- 3252. 회원 정보 관리 서비스 만들기 - 회원가입, 회원 정보 수정 기능
- 3249. 회원 관리 시스템 만들기 - 회원 정보 수정, 비밀번호 변경, 로그아웃

비밀번호 변경

- 3253. 회원 정보 관리 서비스 만들기 - 비밀번호 수정, 회원 탈퇴

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 회원 정보 수정 시 사용하는 Django 내장 폼은?

a) UserCreationForm
b) AuthenticationForm
c) UserChangeForm
d) PasswordChangeForm

2 UserChangeForm을 커스텀할 때, 보여줄 필드만 지정하는 Meta 속성은?

a) exclude 옵션 사용
b) fields 옵션에 원하는 필드만 명시
c) Meta 클래스를 삭제
d) 폼 템플릿에서 직접 HTML 태그 삭제

3 비밀번호 변경 시 사용하는 Django 내장 폼은?

a) UserChangeForm
b) PasswordChangeForm
c) SetPasswordForm
d) AuthenticationForm

4 비밀번호 변경 후 로그아웃되는 현상을 막는 함수는?

a) login(request, user)
b) logout(request)
c) update_session_auth_hash(request, user)
d) save_session(request)

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 비밀번호를 복원이 불가능한 고정 길이 문자열로 바꾸는 과정은?

a) 인코딩 (Encoding)
b) 디코딩 (Decoding)
c) 해싱 (Hashing)
d) 암호화 (Encryption)

6 해시 함수의 특징이 아닌 것은?

a) 입력값이 같으면 항상 같은 결과가 나온다.
b) 해시값으로 원래 입력값을 쉽게 알 수 있다.
c) 입력값이 조금만 달라도 결과는 크게 달라진다.
d) 어떤 길이의 입력이든 고정된 길이로 출력된다.

7 '레인보우 테이블' 공격을 방어하기 위한 기법은?

a) 키 스트레칭 (Key Stretching)
b) 솔트 (Salt) 사용
c) 2단계 인증 (2FA)
d) HTTPS 사용

8 '무차별 대입 공격'의 효율을 떨어뜨리기 위한 기법은?

a) 솔트 (Salt) 사용
b) 키 스트레칭 (Key Stretching)
c) 비밀번호 복잡도 규칙 적용
d) 캡차 (CAPTCHA) 사용

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 Django가 기본으로 사용하는 비밀번호 해싱 알고리즘은?

a) MD5
b) SHA-1
c) PBKDF2
d) bcrypt

10 Django 비밀번호 해시값에 포함되지 않는 정보는?

a) 알고리즘 (algorithm)
b) 반복 횟수 (iterations)
c) 솔트 (salt)
d) 사용자 아이디 (username)

11 비밀번호 초기화 기능을 위해 settings.py에 설정해야 하는 것은?

a) PASSWORD_RESET_TIMEOUT_DAYS
b) EMAIL_BACKEND
c) AUTH_PASSWORD_VALIDATORS
d) LOGIN_REDIRECT_URL

확인 문제

정답 및 해설

1. c) `UserChangeForm` 2. b) fields 옵션에 원하는 필드만 명시 3. b) PasswordChangeForm
4. c) `update_session_auth_hash(request, user)` 5. c) 해싱 (Hashing) 6. b) 해시값으로 원래 입력값을 쉽게 알 수 있다. 7. b) 솔트 (Salt) 사용 8. b) 키 스트레칭 (Key Stretching)

1. UserChangeForm은 등록된 사용자의 정보를 수정하기 위해 Django에서 기본으로 제공하는 ModelForm입니다.
2. 커스텀 폼의 Meta 클래스 안 fields 튜플에 'first_name', 'last_name' 등 원하는 필드 이름만 지정하여 출력 필드를 제어할 수 있습니다.
3. PasswordChangeForm은 사용자가 기존 비밀번호를 입력하고 새 비밀번호로 변경할 수 있도록 돕는 내장 폼입니다.
4. 이 함수는 비밀번호가 변경되었을 때, 새로운 비밀번호 해시값으로 현재 세션 정보를 갱신하여 로그아웃을 방지합니다.
5. 해싱은 임의의 데이터를 고정된 길이의 값으로 변환하는 단방향 과정으로, 비밀번호 저장에 사용됩니다.
6. 해시 함수는 단방향 변환이 특징이며, 결과값(해시)으로 원본 데이터를 역추적하는 것은 거의 불가능합니다.
7. 각 비밀번호에 '솔트'라는 임의의 문자열을 추가하여 해싱하면, 동일한 비밀번호라도 사용자마다 다른 해시값을 갖게 되어 레인보우 테이블을 무력화시킵니다.
8. 해시 연산을 의도적으로 수만 번 이상 반복하여 한번의 비밀번호 검증에 걸리는 시간을 늘림으로써, 무차별 대입 공격의 속도를 극단적으로 늦춥니다.

확인 문제

정답 및 해설

9. c) PBKDF2 10. d) 사용자 아이디 (username) 11. b) EMAIL_BACKEND

9. Django는 솔트와 키 스트레칭을 구현한 PBKDF2 알고리즘을 기본값으로 사용하여 안전하게 비밀번호를 저장합니다.
10. Django의 비밀번호 필드 값은 사용자 아이디는 포함되지 않습니다.
11. 비밀번호 초기화 링크를 이메일로 보내기 때문에, Django가 이메일을 어떻게 처리할지 정의하는 EMAIL_BACKEND 설정이 필요합니다. (콘솔 테스트 포함)

활동 정리

핵심키워드

개념	설명	예시
회원정보 수정 폼	회원정보 수정을 위한 내장 ModelForm	`form = CustomUserChangeForm()`
비밀번호 변경 폼	비밀번호 변경을 위한 내장 Form	`form = PasswordChangeForm(user)`
세션 무효화 방지	비밀번호 변경 후 로그인 유지	`update_session_auth_hash(req, user)`
해시 (Hash)	데이터를 고유한 값으로 변환하는 것	원본 복원이 불가능한 단방향 변환
솔트 (Salt)	해시 전 추가되는 임의의 문자열	레인보우 테이블 공격을 방어함
키 스트레칭	해시 연산을 반복하여 시간 지연	무차별 대입 공격(Brute-force) 방어
비밀번호 초기화	이메일로 비밀번호를 재설정	`include('django.contrib.auth.urls')`

<표2_Django_Authentication 03_핵심키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- 회원정보 수정
  - 회원정보 수정은 User 객체를 갱신(Update)하는 과정
  - Django에 내장된 UserChangeForm은 회원정보 수정을 위한 ModelForm
  - UserChangeForm은 관리자 페이지용이라 사용자의 모든 필드를 보여주므로, 일반 사용자에게는 필요한 필드만 보여주도록 커스텀 폼을 만들어야 함
  - 커스텀 폼(CustomUserChangeForm)은 UserChangeForm을 상속받고, Meta 클래스에서 fields 속성을 조정하여 출력할 필드를 지정

활동 정리

오늘 공부한 내용 요약 및 정리

- 비밀번호 변경
  - 비밀번호 변경은 User 객체의 password 필드를 Update 하는 과정 (기존 비밀번호로 본인을 확인한 뒤 새 비밀번호를 해시해서 저장)
  - PasswordChangeForm을 사용
  - 이 폼은 기존 비밀번호를 확인하고 새 비밀번호가 일치하는지 검증한 후, 비밀번호를 안전하게 암호화하여 갱신
  - 비밀번호를 변경하면 기존 세션이 무효화되어 자동으로 로그아웃
  - update_session_auth_hash 함수를 사용하면 비밀번호 변경 후에도 로그인 상태를 유지할 수 있음

활동 정리

오늘 공부한 내용 요약 및 정리

- 비밀번호 암호화
  - 비밀번호는 데이터베이스 유출 시에도 안전하도록 복원이 불가능한 문자열로 변환하여 저장해야 함
  - 이 과정을 **해시(hash)**라고 하며, 데이터를 고정된 길이의 고유한 값으로 바꾸는 단방향 변환
  - 암호화 공격과 방어 기법
    - 레인보우 테이블 공격: 자주 쓰이는 비밀번호의 해시 값을 미리 계산해 둔 목록을 이용해 비밀번호를 찾아내는 공격 방식
    - 솔트: 레인보우 공격을 막기 위해, 각 사용자의 비밀번호에 고유한 임의의 문자열(솔트)을 추가하여 해시
    - 무차별 대입 공격: 가능한 모든 비밀번호를 하나씩 대입해보는 공격
    - 키 스트레칭: 무차별 공격을 막기 위해, 해시 연산을 수만 번 이상 반복하여 검증 시간을 의도적으로 늘리는 기법
    - Django는 기본적으로 PBKDF2 알고리즘을 사용해 키 스트레칭과 솔트를 적용하며, 비밀번호를 `<algorithm>$<iterations>$<salt>$<hash>` 형식으로 저장함

활동 정리

오늘 공부한 내용 요약 및 정리

- 비밀번호 초기화
  - 비밀번호 초기화는 사용자가 이메일 인증을 통해 비밀번호를 재설정하는 과정
  - Django는 django.contrib.auth.urls를 통해 비밀번호 초기화에 필요한 기능과 URL을 제공
  - 개발 중에는 settings.py에 `EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'` 설정을 추가하여, 실제 이메일 발송 없이 콘솔 창에서 초기화 링크를 확인할 수 있음

활동 정리

내 비밀번호는 안전할까?

- 우리의 비밀번호는 어떻게 저장되고 있을까요?
- 그리고 만약 우리가 만든 서비스가 해킹당해 DB가 통째로 유출된다면 어떻게 될까요?
- 개발자로서, 이 최악의 상황을 어떻게 막아야 할까요?

1. 비밀번호는 우리가 입력한 값으로 저장되고 있을까요?

➢ 아니요! 비밀번호는 해시를 통해 암호화되고, 해시값은 역으로 찾아낼 수 없습니다.

2. 해커가 무수히 많은 시도를 통해 비밀번호를 해킹하려고 하면 어떻게 할까요?

➢ 솔트 기법을 통해 레인보우 테이블 공격을 막고, 키 스트레칭을 통해 무차별 대입 공격을 막아냅니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다